

The `proofgraph` package

Pierre Senellart
`pierre@senellart.com`

2026/05/31 v0.1.0

Abstract

The `proofgraph` package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, requiring no manual annotation. The graph is written as a `Graphviz .dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. `proofgraph` builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`, ...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result. No manual annotation is required.

The package is best explained by a small example of its use. This manual is itself a `proofgraph` document, so the results stated below are really typeset with the package enabled, and the graph shown at the end of this section is the one they produce.

Load `proofgraph` after the theorem-providing package, and declare the usual theorem-like environments:

```
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
\newtheorem{corollary}{Corollary}
```

Then state and prove, as usual, a few results that build on one another:

```
\begin{lemma}\label{lem:even}
  The sum of two even integers is even.
\end{lemma}
\begin{lemma}\label{lem:odd}
  The sum of two odd integers is even.
\end{lemma}
\begin{theorem}\label{thm:parity}
  The sum of two integers of the same parity is even.
```

```

\end{theorem}
\begin{proof}
  By Lemma~\ref{lem:even} and Lemma~\ref{lem:odd}.
\end{proof}
\begin{corollary}\label{cor:double}
  For every integer  $n$ , the number  $2n$  is even.
\end{corollary}
\begin{proof}
  Apply Theorem~\ref{thm:parity} to  $n$  and  $n$ .
\end{proof}

```

We obtain the following results, exactly as without the package:

Lemma 1. *The sum of two even integers is even.*

Lemma 2. *The sum of two odd integers is even.*

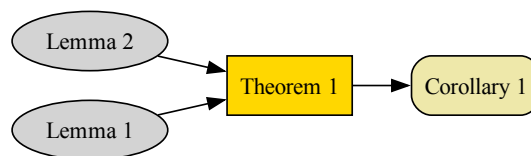
Theorem 1. *The sum of two integers of the same parity is even.*

Proof. By Lemma 1 and Lemma 2. □

Corollary 1. *For every integer n , the number $2n$ is even.*

Proof. Apply Theorem 1 to n and n . □

In addition, `proofgraph` writes out the dependency graph below. Here it is rendered with `Graphviz` and embedded automatically, thanks to the `autorun` option:



Each cross-reference made inside a proof becomes an arrow *into* the result that proof establishes: the two lemmas point to Theorem 1, which in turn points to Corollary 1. Moreover, since this manual loads `hyperref` and `TikZ`, the nodes of the graph above are *clickable*: try clicking one to jump to the result it represents (see `\proofgraph` and the `hyperlinks` option below). The rest of this manual explains how to obtain, tune and extend such graphs.

2 Usage

Load the package after any theorem-providing package (such as `amsthm`) and *before* your `\newtheorem` declarations:

```

\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}

```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with Graphviz:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

`\uses` Inside a proof, `\uses{<label>}` records a dependency on the result labelled `<label>` even when no visible reference is made. A comma-separated list is accepted: `\uses{lem:a,lem:b}`.

A cross-reference in the *optional title* of a result is captured automatically as a dependency: `\begin{theorem}[(generalises~\ref{thm:x})]` in a result labelled `<A>` adds an edge from `<A>` to `thm:x`. This is handy for restatements (e.g. an introduction that previews results with `\begin{result}[(Corollary~\ref{cor:y})]`).

`\proofgraphedge` `\proofgraphedge{<A>}{<labels>}` adds edges recording that result `<A>` depends on the result(s) `<labels>` (a comma-separated list), like a `\uses` but stated outside any proof. This is handy for an “obvious” corollary that relies on earlier results without a `proof` environment: `\proofgraphedge{cor:x}{thm:y,lem:z}`. It may appear anywhere; all labels are resolved when the graph is written.

`\proofof` By default a proof is attached to the most recently started result. A detached proof whose optional title names its result is attached automatically to that result: writing `\begin{proof}` with the optional argument `[of Theorem~\ref{thm:x}]` attaches the proof to `thm:x`. This works for the `amsthm` proof environment and for `proof` defined as a theorem (the Springer classes); deferred proofs (as with `apxproof`) are likewise attached automatically. When none of this applies, `\proofof{<label>}` inside the proof pins it to the result labelled `<label>`.

`\proofgraphtrack` Result environments are detected automatically. Expository environments are excluded from the graph by default (`definition`, `example`, `remark`, `note`). In the preamble, `\proofgraphuntrack{<names>}` excludes further environment types and `\proofgraphtrack{<names>}` forces environments to be tracked (overriding the default exclusions), e.g. `\proofgraphuntrack{observation}` or `\proofgraphtrack{definition}`. Nodes are labelled by the result’s formatted name and number (“Theorem 1”), not by the environment name.

`\proofgraphignore` `\proofgraphignore{<from>}{<to>}` suppresses the edge from `<from>` to `<to>` (e.g. an unwanted forward citation).

`\proofgraphexclude` `\proofgraphexclude{<label>}` removes the result labelled `<label>`, together with all its incident edges, from the graph.

`\proofgraphstyle` `\proofgraphstyle{<env>}{<attrs>}` sets Graphviz node attributes for all results of the environment `<env>`, e.g. `\proofgraphstyle{lemma}{shape=ellipse,color=blue}`.

`\proofgraphstylecite` `\proofgraphstylecite{<attrs>}` sets the Graphviz attributes of the external citation nodes added by the `cite` option (default `shape=note`), e.g. `\proofgraphstylecite{shape=box,style=note}`.

`\proofgraph` In `autorun` mode, `\proofgraph` includes the rendered graph at that point (using the image produced on the previous run). If `hyperref` and `TikZ` are both loaded (and `hyperlinks` is left on), every node is turned into an internal hyperlink to the result it represents: Graphviz writes the node positions to a `-Tplain` file, and `\proofgraph` overlays a transparent `\hyperref` area on each node over the image. This is needed because the link annotations Graphviz puts in the `.pdf` would be discarded by `\includegraphics`. An optional argument is passed to the underlying `\includegraphics`, e.g. `\proofgraph[width=\linewidth]`.

2.1 Options

option **selfloops=remove** (default) drops edges from a result to itself; **selfloops=keep** retains them.

option **autorun** (boolean, default false) runs Graphviz automatically at the end of the run; requires compilation with shell-escape enabled.

option **engine** Graphviz layout engine (default dot).

option **format** Output format for **autorun**/**\proofgraph** (default pdf).

option **hyperlinks** (boolean, default true) makes the nodes of an embedded graph clickable when **hyperref** and **TikZ** are available (see **\proofgraph**). Set **hyperlinks=false** to embed the graph as a plain image.

option **hyperlinks** **direction=usedby** (default) orients an edge from the result that is used to the result that uses it, so an arrow **a->b** reads “a is used by b” (b relies on a). **direction=uses** reverses it (**a->b** means “a uses b”).

option **rankdir** Graph direction (default LR).

option **rankdir** **cite** (boolean, default false) also captures **\cite** inside proofs as dependencies on external work, drawn as distinct nodes. A **\cite** optional note is kept: **\cite[Thm~3]{key}** labels the node “Thm 3, ...”.

option **citelabel=key** (default) labels citation nodes by their BibTeX key; **citelabel=tag** uses the printed citation tag instead (“[1]”, “[Knu74]”, “[Abiteboul et al. 1995]”, ...), recovered from the **.aux** (so it needs two runs). It supports the standard, **hyperref**-wrapped and **natbib** (numeric and author–year) bibliographies; if no clean tag can be recovered (e.g. before the first BibTeX run) it keeps the key.

option **file** Base name of the output file (default **\jobname**).

2.2 Use with apxproof

apxproof moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right result, **proofgraph** relies on the hook **\apxproofhook**, which **apxproof** fires inside each deferred proof. This hook is available in **apxproof v1.4.1** and later; load the two packages in either order and no further setup is needed.

With an *earlier* **apxproof** the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured* (the graph has nodes but no proof edges), because **proofgraph** cannot tell which appendix proof belongs to which result. In that case, add the dependencies manually with **\uses** (and **\proofof** to pin a proof to its result), or upgrade **apxproof**.

3 Limitations

Nodes are created only for environments declared through **\newtheorem** *after* the package is loaded. Results must be labelled for references to them to be resolved. Two compilation runs are required, as for any cross-reference. Theorem titles containing the characters **"** or **** in the node label are not escaped in this version.

4 Implementation

```
1 (*package)
2 \NeedsTeXFormat{LaTeX2e}[2005/12/01]
```

```

3 \ProvidesPackage{proofgraph}
4 [2026/05/31 v0.1.0 Dependency graph of results]

```

4.1 Dependencies

```

5 \RequirePackage{amsthm}
6 \RequirePackage{etoolbox}
7 \RequirePackage{xstring}
8 \RequirePackage{kvoptions}
9 \RequirePackage{graphicx}

```

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite/\cref/\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```

10 \ExplSyntaxOn
11 \cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
12 \ExplSyntaxOff

```

4.2 Options

```

13 \SetupKeyvalOptions{family=pgph,prefix=pgph@}
14 \DeclareStringOption[remove]{selfloops}
15 \DeclareBoolOption{autorun}
16 \DeclareStringOption[dot]{engine}
17 \DeclareStringOption[pdf]{format}
18 \DeclareStringOption[LR]{rankdir}
19 \DeclareStringOption[usedby]{direction}
20 \DeclareBoolOption{cite}
21 \DeclareStringOption[key]{citelabel}
22 \DeclareBoolOption[true]{hyperlinks}
23 \DeclareStringOption{file}
24 \ProcessKeyvalOptions*
25 \ifx\pgph@file\@empty\def\pgph@file{\jobname}\fi

```

4.3 Literal braces

We need catcode-12 braces to write the Graphviz digraph `{ }` block.

```

26 \begingroup
27 \catcode'\[=1 \catcode'\]=2
28 \catcode'\{=12 \catcode'\}=12
29 \gdef\pgph@ob[{}]%
30 \gdef\pgph@cb[{}]%
31 \endgroup

```

4.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```

32 \newcount\pgph@idcnt
33 \def\pgph@nodes{}
34 \def\pgph@edges{}
35 \def\pgph@ignores{}

```

```

36 \def\pgph@excludes{}
37 \let\pgph@curresult\@empty
38 \let\pgph@curproof\@empty

```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the `.aux` file, that the user label maps to the current node id, so references resolve independently of `hyperref`.

```

39 \newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}%
40 \ifcsname pgph@rmap@#2\endcsname\else\global\@namedef{pgph@rmap@#2}{#1}\fi}
41 \newcommand\pgph@setmap[2]{%
42 \edef\pgph@tmpid{#2}%
43 \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
44 \ifcsname pgph@rmap@\pgph@tmpid\endcsname\else
45 \global\expandafter\def\csname pgph@rmap@\pgph@tmpid\endcsname{#1}%
46 \fi
47 \protected@write\@auxout{}{\string\pgph@auxmap{#1}{\pgph@tmpid}}%
48 }

```

4.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```

49 \let\pgph@candidates\@empty
50 \newcommand\pgph@register[1]{%
51 \edef\pgph@thisname{#1}%
52 \IfBeginWith\pgph@thisname{axp@}{-}{%
53 \ifcsname pgph@cand@\pgph@thisname\endcsname\else
54 \global\@namedef{pgph@cand@\pgph@thisname}{-}%
55 \ifx\pgph@candidates\@empty
56 \global\let\pgph@candidates\pgph@thisname
57 \else
58 \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
59 \fi
60 \fi
61 }%
62 }

```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g. the appendix copies produced by `apxproof` and `myproofs`), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```

63 \let\pgph@orig@newtheorem\newtheorem
64 \renewcommand\newtheorem{\ifstar{\pgph@nt@star}{\pgph@nt@plain}}

```

```

65 \newcommand\pgph@nt@plain[1]{\pgph@register{#1}\pgph@orig@newtheorem{#1}}
66 \newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
67 \ifdefined\spnewtheorem
68 \let\pgph@orig@spnewtheorem\spnewtheorem
69 \renewcommand\spnewtheorem{\@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
70 \newcommand\pgph@spnt@plain[1]{\pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
71 \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
72 \fi

```

\proofgraphtrack

\proofgraphuntrack
\proofgraphtrack

Result environments are detected automatically, but the analysis can be tuned.

\proofgraphuntrack

\proofgraphtrack{<names>} forces the listed environments to be tracked, overriding any exclusion; \proofgraphuntrack{<names>} excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the preamble. \pgph@defaultenvs is the list of class-predefined names probed at \begin{document}; \pgph@defaultexclude seeds the exclusion set.

```

73 \newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,definition,%
74 conjecture,claim,fact,remark,example,observation,notation,problem,%
75 question,property,assumption,hypothesis,principle}
76 \newcommand\pgph@defaultexclude{definition,example,remark,note}
77 \newcommand\proofgraphtrack[1]{%
78 \@for\pgph@one:=#1\do{%
79 \csundef\pgph@xtype@\pgph@one}%
80 \global\@namedef\pgph@force@\pgph@one}{}%
81 \expandafter\pgph@register\expandafter{\pgph@one}%
82 }%
83 }
84 \newcommand\proofgraphuntrack[1]{%
85 \@for\pgph@one:=#1\do{%
86 \csundef\pgph@force@\pgph@one}%
87 \global\@namedef\pgph@xtype@\pgph@one}{}%
88 }%
89 }
90 \expandafter\proofgraphuntrack\expandafter{\pgph@defaultexclude}

```

Environments predefined by the document class (acmart, lipics, Springer classes) are declared before the package is loaded, so the wrappers above never see them. At \begin{document} we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```

91 \newcommand\pgph@dohook[1]{%
92 \ifcsname pgph@hooked@#1\endcsname\else
93 \global\@namedef\pgph@hooked@#1}{}%
94 \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
95 \fi
96 }
97 \newcommand\pgph@maybehook[1]{%
98 \ifcsname pgph@force@#1\endcsname
99 \pgph@dohook{#1}%
100 \else
101 \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
102 \fi

```

```

103 }
104 \newcommand\pgph@resolveregistration[1]{%
105   \@for\pgph@one:=#1\do{%
106     \ifcsname\pgph@one\endcsname\expandafter\pgph@register\expandafter{\pgph@one}\fi}%
107   \ifx\pgph@candidates\empty\else
108     \@for\pgph@one:=\pgph@candidates\do{%
109       \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
110   \fi
111 }
112 \AtBeginDocument{%
113   \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}}

```

`\pgph@beginresult` Start a result: allocate an id and record the node. The node is keyed and styled by `\pgph@hooknames` the environment name (third argument, also a fallback label), but displayed by the `\pgph@recordresult` result's *formatted* name ("Theorem", "Lemma"), captured from the heading. We get that name by locally wrapping `\@begintheorem/\@opargbegintheorem` (the hooks every `\amsthm-` and `\newtheorem-` based class routes its heading through), whose first argument is the title. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, Graphviz stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no `pack` trickery.

```

114 \newcommand\pgph@prependnode[1]{%
115   \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
116 }
117 \newcommand\pgph@beginresult[1]{%
118   \global\advance\pgph@idcnt\@ne
119   \xdef\pgph@curresult{\the\pgph@idcnt}%
120   \protected@edef\pgph@tmp{%
121     \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
122   \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
123   \pgph@hooknames
124   \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
125   \let\label\pgph@maplabel
126 }
127 \newcommand\pgph@hooknames{%
128   \ifdefined\@begintheorem
129     \let\pgph@orig@bt\@begintheorem
130     \def\@begintheorem##1##2{\pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
131   \fi
132   \ifdefined\@opargbegintheorem
133     \let\pgph@orig@obt\@opargbegintheorem
134     \def\@opargbegintheorem##1##2##3{%
135       \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
136       \pgph@orig@obt{##1}{##2}{##3}}%
137   \fi
138   \ifdefined\@spbegintheorem
139     \let\pgph@orig@spbt\@spbegintheorem
140     \def\@spbegintheorem##1##2##3##4{%

```



```

141      \pgph@recordresult{##1}{##2}\pgph@orig@spbt{##1}{##2}{##3}{##4}}%
142    \fi
143    \ifdefined\@spopargbegintheorem
144      \let\pgph@orig@spobt\@spopargbegintheorem
145      \def\@spopargbegintheorem##1##2##3##4##5{%
146        \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
147        \pgph@orig@spobt{##1}{##2}{##3}{##4}{##5}}%
148    \fi
149 }

```

`\pgph@scanrefs` A result whose optional title carries a cross-reference, e.g. `\begin{result}[(Corollary~\ref{cor:x})]` is understood to restate or build on that result: we record an edge from the current result to each label referenced in the title. We capture these by typesetting the title once into a discarded box with the reference-capturing commands installed and `\pgph@curproof` pointing at the current result; `\pgph@curproof` is saved and restored, so this affects nothing outside the title.

```

150 \newcommand\pgph@scanrefs[1]{%
151   \ifx\pgph@curresult\@empty\else
152     \begingroup
153       \global\let\pgph@savedit\pgph@curproof
154       \global\let\pgph@curproof\pgph@curresult
155       \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
156       \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
157       \setbox\z@\hbox{#1}%
158       \global\let\pgph@curproof\pgph@savedit
159     \endgroup
160   \fi
161 }

```

The formatted name is the first argument of every heading macro (`\@begintheorem`/`\@opargbegintheorem` for `amsthm`/`\newtheorem`, `\@spbegintheorem`/`\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from `\@currentlabel` rather than from that argument, which some classes pollute (e.g. `lipics` injects `\advance\par@deathcycles\@ne`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably `apxproof`'s forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```

162 \newcommand\pgph@recordresult[2]{%
163   \ifcsname pgph@name@\pgph@curresult\endcsname\else
164     \begingroup
165       \let\protect\@empty
166       \def\axp@forward@link##1##2{##2}%
167       \edef\pgph@tmpname{#1}%
168       \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
169         \pgph@tmpname
170       \global\@namedef{pgph@seen@\pgph@curresult}{}%

```

```

171 \def\pgph@tmphn{#2}%
172 \ifx\pgph@tmphn\@empty\else
173 \edef\pgph@tmpnum{\@currentlabel}%
174 \global\expandafter\let\csname pgph@num@\pgph@curresult\endcsname
175 \pgph@tmpnum
176 \fi
177 \endgroup
178 \fi
179 }

```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing dangerous survives to the emission pass: in particular, under `apxproof` forward-linking, `\@currentlabel` holds `\axp@forward@link{<target>}{<number>}`, whose hyper-link machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

`\pgph@maplabel`

```

180 \newcommand\pgph@maplabel[1]{%
181 \pgph@setmap{#1}{\pgph@curresult}%
182 \ifcsname pgph@num@\pgph@curresult\endcsname\else
183 \ifcsname pgph@seen@\pgph@curresult\endcsname\else
184 \begingroup
185 \let\protect\@empty
186 \def\axp@forward@link##1##2{##2}%
187 \edef\pgph@tmpnum{\@currentlabel}%
188 \global\expandafter\let\csname pgph@num@\pgph@curresult\endcsname\pgph@tmpnum
189 \endgroup
190 \fi
191 \fi
192 \pgph@savetlabel{#1}%
193 }

```

4.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```

194 \newcommand\pgph@adddedge[2]{%
195 \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
196 \expandafter\g@addto@macro\expandafter\pgph@edges\expandafter{\pgph@tmpedge}%
197 }
198 \newcommand\pgph@recordref[1]{%
199 \ifx\pgph@curproof\@empty\else
200 \@for\pgph@one:=#1\do{%
201 \edef\pgph@k{\expandafter\pgph@trimspace\expandafter{\pgph@one}}%
202 \pgph@adddedge{\pgph@curproof}{\pgph@k}%
203 \fi
204 }

```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g. a `\caption`), they do not expand into the `.aux/.lof` and leak internal tokens.

```

205 \protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@saved@ref{#1}}
206 \protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@saved@cref{#1}}
207 \protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@saved@Cref{#1}}
208 \protected\def\pgph@cap@autoref#1{\pgph@recordref{#1}\pgph@saved@autoref{#1}}
209 \protected\def\pgph@cap@eqref#1{\pgph@recordref{#1}\pgph@saved@eqref{#1}}
210 \protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@saved@vref{#1}}
211 \newcommand\pgph@install[1]{%
212   \ifcsname #1\endcsname
213     \expandafter\ifx\csname #1\expandafter\endcsname\csname pgph@cap@#1\endcsname
214     \else
215       \expandafter\let\csname pgph@saved@#1\expandafter\endcsname\csname #1\endcsname
216       \expandafter\let\csname #1\expandafter\endcsname\csname pgph@cap@#1\endcsname
217     \fi
218   \fi
219 }
220 \newcommand\pgph@beginproof{%
221   \let\pgph@curproof\pgph@curresult
222   \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
223   \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
224   \ifpgph@cite\pgph@installcite\fi
225   \pgph@hookproofhead
226 }
```

When the proof environment is itself a theorem-like environment (as with Springer's `\spnewtheorem*{proof}`), its optional title is set by `\@opargbegintheorem/\@spopargbegintheorem`. We wrap these for the duration of the proof so that a title such as `[of Theorem~\ref{thm:x}]` sets `\proofof{thm:x}`, attributing the proof to its result.

```

227 \newcommand\pgph@hookproofhead{%
228   \ifdefined\@opargbegintheorem
229     \let\pgph@orig@obtp\@opargbegintheorem
230     \def\@opargbegintheorem##1##2##3{%
231       \pgph@scanproofarg{##3}\pgph@orig@obtp{##1}{##2}{##3}}%
232   \fi
233   \ifdefined\@spopargbegintheorem
234     \let\pgph@orig@spobtp\@spopargbegintheorem
235     \def\@spopargbegintheorem##1##2##3##4##5{%
236       \pgph@scanproofarg{##3}\pgph@orig@spobtp{##1}{##2}{##3}{##4}{##5}}%
237   \fi
238   \ifdefined\@Opargbegintheorem
239     \let\pgph@orig@Obtp\@Opargbegintheorem
240     \def\@Opargbegintheorem##1##2##3##4{%
241       \pgph@scanproofarg{##2}\pgph@orig@Obtp{##1}{##2}{##3}{##4}}%
242   \fi
243 }
```

`\pgph@scanproofarg` A proof whose optional title names its result (a detached proof) has its result given by that title, e.g. `thm:x` for an optional argument `[of Theorem~\ref{thm:x}]`, rather than by the most recently opened result. We capture this by wrapping the

`proof` environment so that, when an optional title is given, the cross-reference it contains is turned into a `\proofof` (the title is typeset once into a discarded box with `\ref` and friends redefined). Without an optional title the original environment is used unchanged.

```

244 \newcommand\pgph@scanproofarg[1]{%
245   \begingroup
246     \let\protect\@empty
247     \def\ref##1{\proofof{##1}}\def\cref##1{\proofof{##1}}%
248     \def\Cref##1{\proofof{##1}}\def\autoref##1{\proofof{##1}}%
249     \setbox\z@\hbox{#1}%
250   \endgroup
251 }

```

To capture the optional title of a detached `amsthm` proof (`\begin{proof}[of Theorem~\ref{thm:x}]`) we redefine the outer `\proof` to read the optional argument and delegate to the original inner macro (saved as `\pgph@proofinner`), so no recursion occurs. We must do this *only* for a genuine optional-argument environment: a package that captures the proof body verbatim (`apxproof`, `myproofs`) redefines `proof` with no optional argument, and wrapping it would break that capture. We recognise the optional-argument form by the `\@protected@testopt` in its definition. When `proof` is instead a theorem-like environment (Springer `\spnewtheorem*{proof}`) its title is already handled by `\pgph@hookproofhead`, so here we just install the begin-hook.

```

252 \newcommand\pgph@wrapproof{%
253   \ifdefined\proof
254     \edef\pgph@proofmeaning{\meaning\proof}%
255     \edef\pgph@testoptstr{\detokenize{\@protected@testopt}}%
256     \IfSubStr\pgph@proofmeaning\pgph@testoptstr
257       {\pgph@redefproof}%
258       {\AtBeginEnvironment{proof}{\pgph@beginproof}}%
259   \fi
260 }
261 \newcommand\pgph@redefproof{%
262   \expandafter\let\expandafter\pgph@proofinner\csname\string\proof\endcsname
263   \def\proof{\pgph@beginproof
264     \@ifnextchar[{\pgph@proofopt}{\pgph@proofinner[\proofname]}}%
265   \def\pgph@proofopt[##1]{\pgph@scanproofarg{##1}\pgph@proofinner[##1]}%
266 }

```

Packages such as `apxproof` capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture, and the references are only executed at replay time anyway. So we wrap `proof` only when no such package is active; otherwise the deferred-proof capture (below) takes over. This happens at `\begin{document}`, once we know what is loaded.

```

267 \newif\ifpgph@apxproof
268 \AtBeginDocument{%
269   \@ifpackageloaded{apxproof}%
270     {\pgph@apxprooftrue\pgph@apxinit}%
271     {\pgph@wrapproof}%
272 }

```

`\pgph@apxinit` Under `apxproof`, deferred proofs are typeset in the appendix, where their references
`\pgph@apxproofcapture`

finally execute. `apxproof` fires `\apxproofhook` inside each such proof, passing the `axp@r⟨n⟩` label of the proved theorem, which (because `apxproof` also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```

273 \newcommand\pgph@apxinit{\def\apxproofhook##1{\pgph@apxproofcapture{##1}}
274 \newcommand\pgph@apxproofcapture[1]{%
275   \edef\pgph@curproof{\pgph@resolve{#1}}%
276   \ifx\pgph@curproof\@empty\else
277     \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
278     \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
279     \ifpgph@cite\pgph@installcite\fi
280   \fi
281 }
```

`\pgph@hookcite` Optional `\cite` capture: external references become `cite:-`prefixed target labels, drawn later as distinct nodes. A `\cite[⟨note⟩]{⟨key⟩}`, is recorded for the key(s) and shown in the node label as “`⟨note⟩`, `⟨key⟩`”. The note is reduced to a plain string at capture time (fragile wrappers neutralized); the first note seen for a key wins.

```

282 \newcommand\pgph@cap@cite[2][]{%
283   \ifx\pgph@curproof\@empty\else
284     \def\pgph@citenotearg{#1}%
285     \@for\pgph@one:=#2\do{%
286       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
287       \pgph@addedge{\pgph@curproof}{cite:\pgph@k}%
288       \ifx\pgph@citenotearg\@empty\else
289         \expandafter\pgph@recordcitenote\expandafter{\pgph@k}{#1}%
290       \fi}%
291   \fi
292   \pgph@sav@cite{#1}{#2}%
293 }
294 \newcommand\pgph@recordcitenote[2]{%
295   \ifcsname pgph@citenote@#1\endcsname\else
296     \begingroup
297       \pgph@citesanitize
298       \edef\pgph@tmpnote{#2}%
299       \global\expandafter\let\csname pgph@citenote@#1\endcsname\pgph@tmpnote
300     \endgroup
301   \fi
302 }
303 \newcommand\pgph@installcite{%
304   \ifcsname cite\endcsname
305     \let\pgph@sav@cite\cite
306     \let\cite\pgph@cap@cite
307   \fi
308 }
```

4.7 User commands

```

309 \newcommand\uses[1]{\pgph@recordref{#1}}
310 \newcommand\proofof[1]{%
311   \ifcsname pgph@map@#1\endcsname
```

```

312 \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
313 \fi
314 }
315 \newcommand\proofgraphedge[2]{%
316 \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
317 }
318 \newcommand\pgph@labeledge[2]{%
319 \edef\pgph@ef{\pgph@resolve{#1}}%
320 \ifx\pgph@ef\@empty\else
321 \@for\pgph@one:=#2\do{%
322 \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
323 \pgph@plainedge{\pgph@ef}{\pgph@k}}%
324 \fi
325 }
326 \newcommand\proofgraphignore[2]{%
327 \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}%
328 }
329 \newcommand\proofgraphexclude[1]{%
330 \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}%
331 }
332 \newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}

```

External citation nodes (option cite) share one style, held in `\pgph@citestyle` and overridable with `\proofgraphstylecite`; it defaults to `shape=note`.

```

333 \newcommand\pgph@citestyle{shape=note}
334 \newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}

```

4.8 Resolution helpers

These run at end of document, after the `.aux` has populated the map.

```

335 \newcommand\pgph@resolve[1]{%
336 \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
337 }
338 \newcommand\pgph@ignorerule[2]{%
339 \edef\pgph@f{\pgph@resolve{#1}}\edef\pgph@t{\pgph@resolve{#2}}%
340 \ifx\pgph@f\@empty\else\ifx\pgph@t\@empty\else
341 \@namedef{pgph@ig@\pgph@f @\pgph@t}{}%
342 \fi\fi
343 }
344 \newcommand\pgph@excluderule[1]{%
345 \edef\pgph@x{\pgph@resolve{#1}}%
346 \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
347 }

```

4.9 Emission

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`cite:`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g. a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{<env>}` overrides this, so a deliberately unnumbered environment is still drawn.

```

348 \newcommand\pgph@node[3]{%

```

```

349 \ifcsname pgph@ex@#1\endcsname\else
350 \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname\csname pgph@num@#1\endcsname\fi}%
351 \ifx\pgph@nm\@empty
352 \ifcsname pgph@force@#2\endcsname
353 \pgph@emitnode{#1}{#2}{#3}%
354 \else
355 \@namedef{pgph@ex@#1}{}%
356 \fi
357 \else
358 \pgph@emitnode{#1}{#2}{#3}%
359 \fi
360 \fi
361 }
362 \newcommand\pgph@emitnode[3]{%
363 \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
364 , \csname pgph@style@#2\endcsname\fi}%
365 \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
366 \csname pgph@name@#1\endcsname\else#3\fi}%
367 \immediate\write\pgph@out{%
368 \space\space"n#1" [label="\pgph@dn\space\pgph@nm"\pgph@s];}%
369 \ifcsname pgph@rmap@#1\endcsname
370 \immediate\write\pgph@mapout{%
371 \string\pgph@nodemap{n#1}{\csname pgph@rmap@#1\endcsname}}}%
372 \fi
373 }

```

`\pgph@edge` Emit one edge, resolving the target label to an id and applying the self-loop, ignore and exclude filters. Targets beginning with `cite:` denote external nodes.

```

374 \newcommand\pgph@edge[2]{%
375 \IfBeginWith{#2}{cite:}%
376 {\pgph@citeedge{#1}{#2}}%
377 {\pgph@plainedge{#1}{#2}}%
378 }
379 \newcommand\pgph@plainedge[2]{%
380 \edef\pgph@t{\pgph@resolve{#2}}%
381 \ifx\pgph@t\@empty\else
382 \ifcsname pgph@ex@#1\endcsname\else
383 \ifcsname pgph@ex@\pgph@t\endcsname\else
384 \ifcsname pgph@ig@#1@\pgph@t\endcsname\else
385 \ifboolexpr{ test {\ifnumequal{#1}{\pgph@t}}
386 and test {\ifdefstring{\pgph@selfloops}{remove}} }%
387 {\pgph@emitedge{#1}{n\pgph@t}}%
388 \fi\fi\fi
389 \fi
390 }

```

`\pgph@emitedge` The relation recorded is always “the result whose proof we are in (the first argument) uses the target (the second)”. The `direction` option chooses the arrow orientation: `direction=usedby` (default) draws *target to result*, so an arrow `a->b` reads “a is used by b” (i.e. b relies on a); `direction=uses` draws *result to target* (“a uses b”). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. `\pgph@writeedge` deduplicates.

```

391 \newcommand\pgph@emitedge[2]{%

```

```

392 \ifdefstring{\pgph@direction}{uses}%
393   {\pgph@writeedge{#1}{#2}}%
394   {\pgph@writeedge{#2}{#1}}%
395 }
396 \newcommand\pgph@writeedge[2]{%
397   \ifcsname pgph@drawn@#1@#2\endcsname\else
398     \@namedef{pgph@drawn@#1@#2}{}%
399     \immediate\write\pgph@out{\space\space"#1" -> "#2";}%
400   \fi
401 }

```

`\pgph@citedge` A `cite:key` target: declare the external node once (keyed, as a stable id, by the `\pgph@setcitelabel` BIBTEX key), then draw the edge. The node *label* is built by `\pgph@setcitelabel`: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the “[42]”/“[Knu74]” text) recovered from the `\b@key` macro that `\bibcite` writes to the .aux (so two runs are needed, and it falls back to the key when no tag is known, e.g. on the first run or with citation packages that do not populate `\b@key`). A `\cite` note, when present, is prepended as “*(note)*, ...”.

```

402 \newcommand\pgph@citedge[2]{%
403   \ifcsname pgph@ex@#1\endcsname\else
404     \StrBehind{#2}{cite:}[\pgph@key]%
405     \ifcsname pgph@extnode@\pgph@key\endcsname\else
406       \@namedef{pgph@extnode@\pgph@key}{}%
407       \expandafter\pgph@setcitelabel\expandafter{\pgph@key}%
408       \immediate\write\pgph@out{%
409         \space\space"c@\pgph@key" [label="\pgph@clabel",\pgph@citestyle];}%
410     \fi
411     \pgph@emittedge{n#1}{c@\pgph@key}%
412   \fi
413 }
414 \edef\pgph@obraces{\expandafter\@gobble\string\{ }
415 \newif\ifpgph@natnum
416 \newcommand\pgph@nil{ }

```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@key` yields plain text: `hyperref's \hyper@link` (keep its printed last argument) and the `boxing/penalty/\ignorespaces` noise that `natbib` bakes into author lists.

```

417 \newcommand\pgph@citesanitize{%
418   \let\protect\@empty
419   \def\hyper@link[##1]##2##3##4{##4}%
420   \let\unskip\@empty \let\ignorespaces\@empty \let\unhbox\@empty
421   \let\penalty\@gobble \let\@M\@empty \let\voidb@x\@empty
422   \let\nobreakspace\space \let~\space
423   \def\hbox##1{##1}\def\mbox##1{##1}%
424 }

```

A `natbib \b@key` is `{(num)}{(year)}{(short)}{(long)}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```

425 \def\pgph@natpick#1#2#3#4\pgph@nil{%
426   \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi}
427 \newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x{}{}{}\pgph@nil}
428 \newcommand\pgph@setcitelabel[1]{%
429   \edef\pgph@cbase{#1}%

```



```

430 \ifdefstring{\pgph@citelabel}{tag}{%
431   \ifcsname b@#1\endcsname
432     \global\let\pgph@cbaseg\relax
433     \begingroup
434       \pgph@citesanitize
435       \edef\pgph@x{\csname b@#1\endcsname}%
436       \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
437       \IfBeginWith\pgph@dx\pgph@obrace{%
438         \pgph@natnumfalse
439         \@ifundefined{ifNAT@numbers}{-}{%
440           \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
441             \pgph@natnumtrue\fi}%
442         \pgph@natdo
443         \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
444         -}%
445       \ifx\pgph@x\@empty\else
446         \IfSubStr\pgph@dx\@backslashchar{-}{%
447           \global\let\pgph@cbaseg\pgph@x}%
448       \fi
449     \endgroup
450     \ifx\pgph@cbaseg\relax\else\edef\pgph@cbase{\pgph@cbaseg}\fi
451   \fi
452 }{}%
453 \edef\pgph@clabel{%
454   \ifcsname pgph@citenote@#1\endcsname
455     \csname pgph@citenote@#1\endcsname, \fi
456   \pgph@cbase}%
457 }

```

`\pgph@emit` Open the file, resolve editing rules, write nodes then edges, close, and optionally run Graphviz.

```

458 \newwrite\pgph@out
459 \newwrite\pgph@mapout
460 \newcommand\pgph@emit{%
461   \immediate\openout\pgph@out=\pgph@file.dot\relax
462   \immediate\openout\pgph@mapout=\pgph@file-proofgraph.map\relax
463   \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
464   \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
465   \begingroup
466     \pgph@ignores
467     \pgph@excludes
468     \pgph@nodes
469     \pgph@edges
470   \endgroup
471   \immediate\write\pgph@out{\pgph@cb}%
472   \immediate\closeout\pgph@out
473   \immediate\closeout\pgph@mapout
474   \ifpgph@autorun
475     \immediate\write18{\pgph@engine\space -T\pgph@format\space
476       \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
477     \immediate\write18{\pgph@engine\space -Tplain\space
478       \pgph@file.dot -o \pgph@file-proofgraph.plain}%
479   \fi
480 }

```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably `apxproof`, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```
481 \AtBeginDocument{\AtEndDocument{\pgph@emit}}
```

If TikZ is available, load its `calc` library now (at `\begin{document}`, where category codes are normal) so that the hyperlink overlay in `\proofgraph` can use it without reading a file mid-document.

```
482 \AtBeginDocument{\@ifpackageloaded{tikz}{\usetikzlibrary{calc}}{}}
```

`\pgphnodemap` One entry of the node-to-label map written by `\pgph@emit`: it records, for a Graphviz node identifier, the user label of the result it represents. The map is read back when the graph is embedded, to turn each node into a hyperlink.

```
483 \newcommand\pgphnodemap[2]{\global\@namedef{pgph@lbl@#1}{#2}}
```

`\proofgraph` Include the rendered graph (from a previous run) in `autorun` mode. When `hyperref` and `TikZ` are both available (and `hyperlinks` is not switched off), each node is made into an internal hyperlink to the result it represents: `Graphviz` produces a `.pdf` whose link annotations would be lost by `\includegraphics` (they sit inside a form `XOBJECT`), so instead we overlay transparent `\hyperref` areas at the node positions, which we read from the `-Tplain` output. Without those packages we fall back to a plain inclusion.

```
484 \newif\ifpgph@canlink
485 \newsavebox\pgph@imgbox
486 \newread\pgph@plainin
487 \def\pgph@stop{}
488 \newcommand\proofgraph[1][{}]{%
489   \IfFileExists{\pgph@file-proofgraph.\pgph@format}%
490     {\pgph@includegraph{#1}}%
491     {\PackageWarning{proofgraph}{Rendered graph
492       \pgph@file-proofgraph.\pgph@format\space not found; compile with
493       shell-escape and the autorun option, then re-run}}%
494 }
495 \newcommand\pgph@includegraph[1]{%
496   \pgph@canlinkfalse
497   \ifpgph@hyperlinks
498     \@ifpackageloaded{hyperref}%
499       {\@ifpackageloaded{tikz}%
500         {\IfFileExists{\pgph@file-proofgraph.plain}{\pgph@canlinktrue}{}}{}}}%
501   \fi
502   \ifpgph@canlink
503     \pgph@graphlinked{#1}%
504   \else
505     \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}%
506   \fi
507 }
```

The linked version. We read the node-to-label map under safe category codes (a label may contain `_` or, under `babel`, an active `:`), measure the rendered image, place it in a `TikZ` node, then read the `-Tplain` file and drop one transparent

\hyperref box on each labelled node. Positions are expressed as fractions of the bounding box, so they follow whatever size the image is scaled to.

```

508 \newcommand\pgph@graphlinked[1]{%
509   \begin{group}
510     \catcode'\:=12 \catcode'\_:=12 \catcode'\;=12 \catcode'\!=12
511     \catcode'\?=12 \catcode'\&=12 \catcode'\^=12 \catcode'\~=12
512     \InputIfFileExists{\pgph@file-proofgraph.map}{\}{}%
513   \end{group}
514   \sbox\pgph@imgbox{\includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
515   \edef\pgph@imgw{\the\wd\pgph@imgbox}%
516   \edef\pgph@imgh{\the\dimexpr\ht\pgph@imgbox+\dp\pgph@imgbox\relax}%
517   \begin{tikzpicture}
518     \node[inner sep=\z@,outer sep=\z@](pgph@P){\usebox\pgph@imgbox};
519     \openin\pgph@plainin=\pgph@file-proofgraph.plain\relax
520     \pgph@loopplain
521     \closein\pgph@plainin
522   \end{tikzpicture}%
523 }
524 \newcommand\pgph@loopplain{%
525   \unless\ifeof\pgph@plainin
526     \read\pgph@plainin to \pgph@line
527     \pgph@procline
528     \expandafter\pgph@loopplain
529   \fi
530 }
531 \newcommand\pgph@procline{%
532   \IfBeginWith{\pgph@line}{graph }%
533     {\expandafter\pgph@dograph\pgph@line\pgph@stop}%
534     {\IfBeginWith{\pgph@line}{node }%
535       {\expandafter\pgph@donode\pgph@line\pgph@stop}{}}%
536 }
537 \def\pgph@dograph graph #1 #2 #3 #4\pgph@stop{%
538   \def\pgph@gw{#2}\def\pgph@gh{#3}}
539 \def\pgph@donode node #1 #2 #3 #4 #5 #6\pgph@stop{%
540   \ifcsname pgph@lbl@#1\endcsname
541     \pgfmathsetmacro\pgph@fx{#2/\pgph@gw}%
542     \pgfmathsetmacro\pgph@fy{#3/\pgph@gh}%
543     \pgfmathsetlengthmacro\pgph@bw{#4/\pgph@gw*\pgph@imgw*0.9}%
544     \pgfmathsetlengthmacro\pgph@bh{#5/\pgph@gh*\pgph@imgh*0.85}%
545     \edef\pgph@thislbl{\csname pgph@lbl@#1\endcsname}%
546     \node[anchor=center] at ($(\pgph@P.south west)!\pgph@fx!(pgph@P.south east)$)%
547       !\pgph@fy!($(\pgph@P.north west)!\pgph@fx!(pgph@P.north east)$)$)
548     {\hyperref[\pgph@thislbl]{\phantom{\rule{\pgph@bw}{\pgph@bh}}}};%
549   \fi
550 }
551 \end{package}

```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols	B	<code>\IfSubStr</code> 256, 446
<code>\!</code> 510	<code>\begin</code> 517	<code>\iftrue</code> 440
<code>\&</code> 511	C	<code>\ignorespaces</code> 420
<code>\:</code> 510	<code>\catcode</code> 27, 28, 510, 511	<code>\includegraphics</code> 505, 514
<code>\;</code> 510	<code>\cite</code> 305, 306	<code>\InputIfFileExists</code> . 512
<code>\?</code> 511	<code>\closein</code> 521	
<code>\@M</code> 421	<code>\Cref</code> 248	J
<code>\@Opargbegintheorem</code> 238, 239, 240	<code>\cref</code> 247	<code>\jobname</code> 25
<code>\@auxout</code> 47	<code>\cs</code> 11	L
<code>\@backslashchar</code> ... 446	<code>\csundef</code> 79, 86	<code>\label</code> 124, 125
<code>\@begintheorem</code> 128, 129, 130	D	M
<code>\@currentlabel</code> 173, 187	<code>\DeclareBoolOption</code> 15, 20, 22	<code>\mbox</code> 423
<code>\@ifpackageloaded</code> 269, 482, 498, 499	<code>\DeclareStringOption</code> 14, 16, 17, 18, 19, 21, 23	<code>\meaning</code> 254
<code>\@ifundefined</code> 439	<code>\detokenize</code> 255, 436, 443	N
<code>\@namedef</code> 39, 40, 54, 80, 87, 93, 170, 332, 341, 346, 355, 398, 406, 483	<code>\dimexpr</code> 516	<code>\newcount</code> 32
<code>\@opargbegintheorem</code> 132, 133, 134, 228, 229, 230	<code>\dp</code> 516	<code>\newif</code> 267, 415, 484
<code>\@protected@testopt</code> 255	E	<code>\newread</code> 486
<code>\@spbegintheorem</code> 138, 139, 140	<code>\end</code> 522	<code>\newsavebox</code> 485
<code>\@spopargbegintheorem</code> 143, 144, 145, 233, 234, 235	<code>\ExplSyntaxOff</code> 12	<code>\newtheorem</code> 63, 64
<code>\[</code> 27	<code>\ExplSyntaxOn</code> 10	<code>\nobreakspace</code> 422
<code>\{</code> 28, 414	H	<code>\node</code> 518, 546
<code>\}</code> 28	<code>\hbox</code> 157, 249, 423	<code>\noexpand</code> 121, 195
<code>\]</code> 27	<code>\ht</code> 516	O
<code>\^</code> 511	<code>\hyper@@link</code> 419	P
<code>_</code> 510	<code>\hyperref</code> 548	<code>\PackageWarning</code> ... 491
<code>\~</code> 511	I	<code>\penalty</code> 421
A	<code>\IfBeginWith</code> ... 52, 375, 437, 532, 534	<code>\pgfmithsetlengthmacro</code> 543, 544
<code>\apxproofhook</code> 273	<code>\ifboolexpr</code> 385	<code>\pgfmithsetmacro</code> 541, 542
<code>\AtBeginDocument</code> 112, 268, 481, 482	<code>\ifdefstring</code> 386, 392, 430	<code>\pgph@addedge</code> 194, 202, 287
<code>\AtBeginEnvironment</code> 94, 258	<code>\ifeof</code> 525	<code>\pgph@apxinit</code> . 270, 273
<code>\AtEndDocument</code> 481	<code>\IfFileExists</code> . 489, 500	<code>\pgph@apxproofcapture</code> 273
<code>\autoref</code> 248	<code>\ifnumequal</code> 385	<code>\pgph@apxprooftrue</code> . 270
<code>\axp@forward@link</code> 166, 186	<code>\ifpgph@apxproof</code> .. 267	<code>\pgph@auxmap</code> 39
	<code>\ifpgph@autorun</code> ... 474	<code>\pgph@beginproof</code> 205, 258, 263
	<code>\ifpgph@canlink</code> 484, 502	<code>\pgph@beginresult</code> 94, 114
	<code>\ifpgph@cite</code> .. 224, 279	
	<code>\ifpgph@hyperlinks</code> . 497	
	<code>\ifpgph@natnum</code> 415, 426	

<code>\pgph@bh</code>	544, 548	<code>\pgph@emitnode</code>		<code>\pgph@natdo</code> . . .	427, 442
<code>\pgph@bw</code>	543, 548		353, 358, 362	<code>\pgph@natnumfalse</code> .	438
<code>\pgph@candidates</code> 49,		<code>\pgph@engine</code> . .	475, 477	<code>\pgph@natnumtrue</code> . .	441
55, 56, 58, 107, 108		<code>\pgph@excluderule</code> .		<code>\pgph@natpick</code> .	425, 427
<code>\pgph@canlinkfalse</code> .	496		330, 344	<code>\pgph@nil</code> .	416, 425, 427
<code>\pgph@canlinktrue</code> .	500	<code>\pgph@excludes</code>		<code>\pgph@nm</code> . .	350, 351, 368
<code>\pgph@cap@autoref</code> .	208		36, 330, 467	<code>\pgph@node</code> . . .	121, 348
<code>\pgph@cap@cite</code> 282, 306		<code>\pgph@f</code> . . .	339, 340, 341	<code>\pgph@nodes</code> 33, 115, 468	
<code>\pgph@cap@Cref</code>	207	<code>\pgph@file</code> .	25, 461,	<code>\pgph@nt@plain</code> . .	64, 65
<code>\pgph@cap@cref</code>	206		462, 476, 478,	<code>\pgph@nt@star</code> . . .	64, 66
<code>\pgph@cap@eqref</code> . . .	209		489, 492, 500,	<code>\pgph@ob</code>	29, 463
<code>\pgph@cap@ref</code>	205		505, 512, 514, 519	<code>\pgph@obrace</code> . .	414, 437
<code>\pgph@cap@vref</code>	210	<code>\pgph@format</code> 475, 476,		<code>\pgph@one</code> 78, 79, 80,	
<code>\pgph@cb</code>	30, 471		489, 492, 505, 514	81, 85, 86, 87,	
<code>\pgph@cbase</code> 429, 450, 456		<code>\pgph@fx</code> . .	541, 546, 547	105, 106, 108,	
<code>\pgph@cbaseg</code>		<code>\pgph@fy</code>	542, 547	109, 200, 201,	
432, 447, 450		<code>\pgph@gh</code> . .	538, 542, 544	285, 286, 321, 322	
<code>\pgph@citeedge</code> 376, 402		<code>\pgph@graphlinked</code> .		<code>\pgph@orig@bt</code> .	129, 130
<code>\pgph@citelabel</code> . . .	430		503, 508	<code>\pgph@orig@newtheorem</code>	
<code>\pgph@citenotearg</code> .		<code>\pgph@gw</code> . .	538, 541, 543	63, 65, 66	
284, 288		<code>\pgph@hookcite</code>	282	<code>\pgph@orig@obt</code> 133, 136	
<code>\pgph@citesanitize</code> .		<code>\pgph@hooknames</code> . . .	114	<code>\pgph@orig@Obtp</code> 239, 241	
297, 417, 434		<code>\pgph@hookproofhead</code>		<code>\pgph@orig@obtp</code> 229, 231	
<code>\pgph@citestyle</code> . . .			225, 227	<code>\pgph@orig@spbt</code> 139, 141	
333, 334, 409		<code>\pgph@idcnt</code> 32, 118, 119		<code>\pgph@orig@spnewtheorem</code>	
<code>\pgph@clabel</code> . .	409, 453	<code>\pgph@ignorerule</code> . .		68, 70, 71	
<code>\pgph@curproof</code>			327, 338	<code>\pgph@orig@spobt</code> . .	
38, 153,		<code>\pgph@ignores</code>		144, 147	
154, 158, 199,			35, 327, 466	<code>\pgph@orig@spobtp</code> .	
202, 221, 275,		<code>\pgph@imgbox</code> . .	485,	234, 236	
276, 283, 287, 312			514, 515, 516, 518	<code>\pgph@out</code> .	367, 399,
<code>\pgph@curresult</code> 37,		<code>\pgph@imggh</code> . . .	516, 544	408, 458, 461,	
119, 121, 151,		<code>\pgph@imgw</code> . . .	515, 543	463, 464, 471, 472	
154, 163, 168,		<code>\pgph@includegraph</code> .		<code>\pgph@plainedge</code> . . .	
170, 174, 181,			490, 495	323, 377, 379	
182, 183, 188, 221		<code>\pgph@install</code>		<code>\pgph@plainin</code> .	486,
<code>\pgph@defaultenvs</code> .		155, 156, 211,		519, 521, 525, 526	
73, 113		222, 223, 277, 278		<code>\pgph@prependnode</code> .	
<code>\pgph@defaulttexclude</code>		<code>\pgph@installcite</code> .		114, 122	
76, 90			224, 279, 303	<code>\pgph@procline</code> 527, 531	
<code>\pgph@direction</code> . . .	392	<code>\pgph@k</code> 201, 202, 286,		<code>\pgph@proofinner</code> . .	
<code>\pgph@dn</code>	365, 368	287, 289, 322, 323		262, 264, 265	
<code>\pgph@dograph</code> .	533, 537	<code>\pgph@key</code> .	404, 405,	<code>\pgph@proofmeaning</code> .	
<code>\pgph@dohook</code> 91, 99, 101		406, 407, 409, 411		254, 256	
<code>\pgph@donode</code> . .	535, 539	<code>\pgph@labeledge</code> 316, 318		<code>\pgph@proofopt</code> 264, 265	
<code>\pgph@dx</code>		<code>\pgph@line</code> . . .	526,	<code>\pgph@rankdir</code>	464
436, 437, 443, 446			532, 533, 534, 535	<code>\pgph@recordcitenote</code>	
<code>\pgph@edge</code> . . .	195, 374	<code>\pgph@loopplain</code> . . .		289, 294	
<code>\pgph@edges</code>			520, 524, 528	<code>\pgph@recordref</code> 198,	
34, 196, 316, 469		<code>\pgph@maplabel</code>		205, 206, 207,	
<code>\pgph@ef</code> . .	319, 320, 323		124, 125, 180	208, 209, 210, 309	
<code>\pgph@emit</code>	458	<code>\pgph@mapout</code>		<code>\pgph@recordresult</code> .	114
<code>\pgph@emitedge</code>			370, 459, 462, 473	<code>\pgph@redefproof</code> . .	
387, 391, 411		<code>\pgph@maybehook</code> 97, 109		257, 261	

<code>\pgph@register</code>	<code>\pgph@thisname</code> . 51, 49, 65, 70, 81, 106	<code>\protected</code> 205, 206, 207, 208, 209, 210
<code>\pgph@resolve</code>	<code>\pgph@tmp</code> 120, 122	<code>\protected@edef</code> 120, 195
. 275, 319, 335, 339, 345, 380	<code>\pgph@tmpedge</code> . 195, 196	<code>\protected@write</code> . . 47
<code>\pgph@resolveregistration</code>	<code>\pgph@tmpn</code> 171, 172	
. 104, 113	<code>\pgph@tmpid</code>	R
<code>\pgph@s</code> 363, 368	. 42, 43, 44, 45, 47	<code>\read</code> 526
<code>\pgph@samed@autoref</code> 208	<code>\pgph@tmpname</code> . 167, 169	<code>\ref</code> 247
<code>\pgph@samed@cite</code> . .	<code>\pgph@tmpnote</code> . 298, 299	<code>\rule</code> 548
. 292, 305	<code>\pgph@tmpnum</code>	
<code>\pgph@samed@Cref</code> . . 207	. 173, 175, 187, 188	S
<code>\pgph@samed@cref</code> . . 206	<code>\pgph@trimsp</code>	<code>\sbox</code> 514
<code>\pgph@samed@eqref</code> . 209	. 10, 201, 286, 322	<code>\setbox</code> 157, 249
<code>\pgph@samed@ref</code> . . . 205	<code>\pgph@wrapproof</code> 244, 271	<code>\SetupKeyvalOptions</code> 13
<code>\pgph@samed@vref</code> . . 210	<code>\pgph@writeedge</code> . . . 391	<code>\spnewtheorem</code> 67, 68, 69
<code>\pgph@samedcurproof</code>	<code>\pgph@x</code> . . . 345, 346, 426, 427, 435,	<code>\StrBehind</code> 404
. 153, 158	436, 443, 445, 447	
<code>\pgph@samedlabel</code> . .	<code>\pgphnodemap</code> . . 371, 483	T
. 124, 192	<code>\phantom</code> 548	<code>\tl</code> 11
<code>\pgph@scanproofarg</code> .	<code>\ProcessKeyvalOptions</code>	U
. 231, 236, 241, 244	 24	<code>\unexpanded</code> 115
<code>\pgph@scanrefs</code>	<code>\proof</code> 253, 254, 262, 263	<code>\unhbox</code> 420
. 135, 146, 150	<code>\proofgraph</code> 3, 484	<code>\unless</code> 525
<code>\pgph@selfloops</code> . . . 386	<code>\proofgraphedge</code> . 3, 315	<code>\unskip</code> 420
<code>\pgph@setcitelabel</code> . 402	<code>\proofgraphexclude</code> .	<code>\usebox</code> 518
<code>\pgph@setmap</code> . . . 41, 181	 3, 329	<code>\uses</code> 3, 309
<code>\pgph@spnt@plain</code> 69, 70	<code>\proofgraphignore</code> 3, 326	<code>\usetikzlibrary</code> . . . 482
<code>\pgph@spnt@star</code> . 69, 71	<code>\proofgraphstyle</code> 3, 332	
<code>\pgph@stop</code> 487, 533, 535, 537, 539	<code>\proofgraphstylecite</code>	V
<code>\pgph@t</code> . . . 339, 340, 341, 380, 381, 383, 384, 385, 387	 3, 334	<code>\voidb@x</code> 421
<code>\pgph@testoptstr</code> . .	<code>\proofgraphtrack</code> 3, 7, 73	W
. 255, 256	<code>\proofgraphuntrack</code> .	<code>\wd</code> 515
<code>\pgph@thislbl</code> . 545, 548	 3, 7, 73	Z
	<code>\proofname</code> 264	<code>\z@</code> 157, 249, 518
	<code>\proofof</code> 3, 247, 248, 310	
	<code>\protect</code>	
	. 165, 185, 246, 418	

Change History

v0.1.0

General: Initial version 1